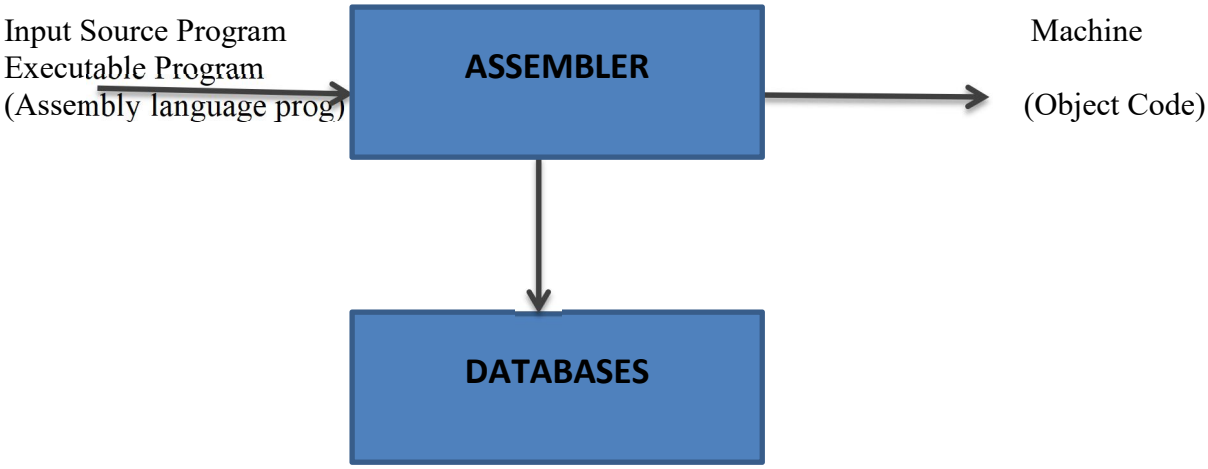
Experiment No.1
Design & Implementation of Pass-1 of Two Pass Assembler
Date of Performance:09/03/26
Date of Submission:16/03/26

Aim: Design & Implementation of Pass-1 of Two Pass Assembler.

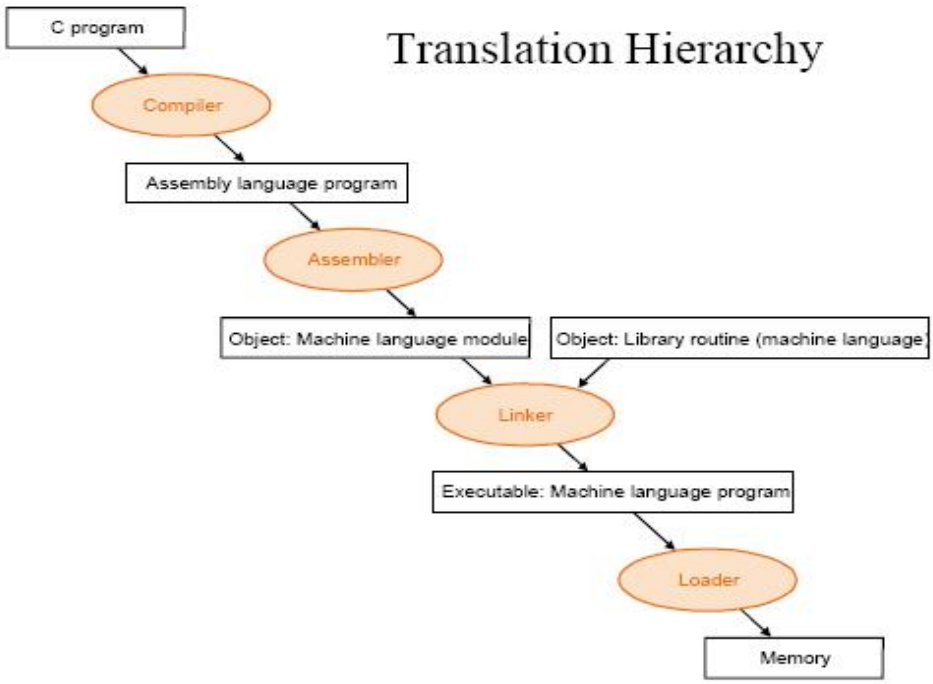
Objective: To study and implement Pass 1 of two pass assembler.

Theory:

An assembler is a program that accepts as input an assembly language program (source) and produces its machine language equivalent (object code) along with the information for the loader.



Relevance of Different system programs:



General Design Steps:

1. Specify the problem
2. Specify data structures
3. Define format of data structures
4. Specify algorithm
5. Look for modularity [capability of one program to be subdivided into independent programming units.]
6. Repeat 1 through 5 on modules

In following design IBM360 machine architecture is considered for reference.

Students should design 8086 / 8088 machine architecture.

1. Specify the problem

Pass1: Define symbols & literals.

1) Determine length of m/c instruction [MOTGET1]
2) Keep track of Location Counter [LC]
3) Remember values of symbols [STSTO]
4) Process some pseudo ops[EQU,DS etc] [POTGET1]
5) Remember Literals [LITSTO]

2. Data structure

Pass1: Databases

1. Input source program
2. "LC" location counter used to keep track of each instructions address.
3. M/c operation table (MOT) [Symbolic mnemonic & length]
4. Pseudo operation table [POT], [Symbolic mnemonic & action]
5. Symbol Table (ST) to store each lable & it's value.
6. Literal Table (LT), to store each literal (variable) & it's location.
7. Copy of input to used later by PASS-2.

3. Format of Data Structures

1. Machine Operation Table (MOT):

The op-code is the key and it's value is the binary op code equivalent, which is used for use in generating machine code.

The instruction length is stored for updating the location counter.

Instruction format is use in forming the m/c language equivalent.

6 bytes entry

Mnemonic Op-code (4 byte) character	Binary Op-code (1 byte) Hex	Instruction length (2 bits) (binary)	Instruction format (3 bits) (binary)	Not used in this design (3 bits)
"Abbb"	5A	10	001	-
"LOAD"	4A	10	001	-
"MOVb"	5E	01	000	-
"MVIb"	1E	11	100	-

b→ represents blank

Codes used-

Instruction length

01 = 1 half words

10 = 2 half words

11 = 3 half words

Instruction

000 - RR

001 – RX

010 – RS

011 – SI

100 – SS

2. Pseudo Operation Table (POT):

8 bytes / entry

Pseudo-op (5-bytes) (character)	Address of routine to process pseudo-op 3 bytes = 24 bit
"EQUbb"	P1EQG
"ENDbb"	P1END
"START"	P1START
"USING"	P1USING

These are presumably labels of routines in pass1

3. Symbol table & Literal table:-

14 bytes / entry

Symbol 8 byte character	Value (4 byte) Hex	Length 1 byte Hex	Relocation 1 byte character
"LOOPbbbb"	0000	01	"R"
"JOHNbbbb"	0010	04	"R"

Relation – R- Relative
A- Absolute (i.e. does not change)

4. Base table (BT):

4 bytes / entry

	Availability Indicator 1 byte Character	Designated relative – address contents of base register (3 bytes = 24bit address) Hex	
1	"N"	-	15 entries
2	"N"	-	
3	"N"	-	
.	:	-	
.	:	-	
15	"Y"	00 00 00	

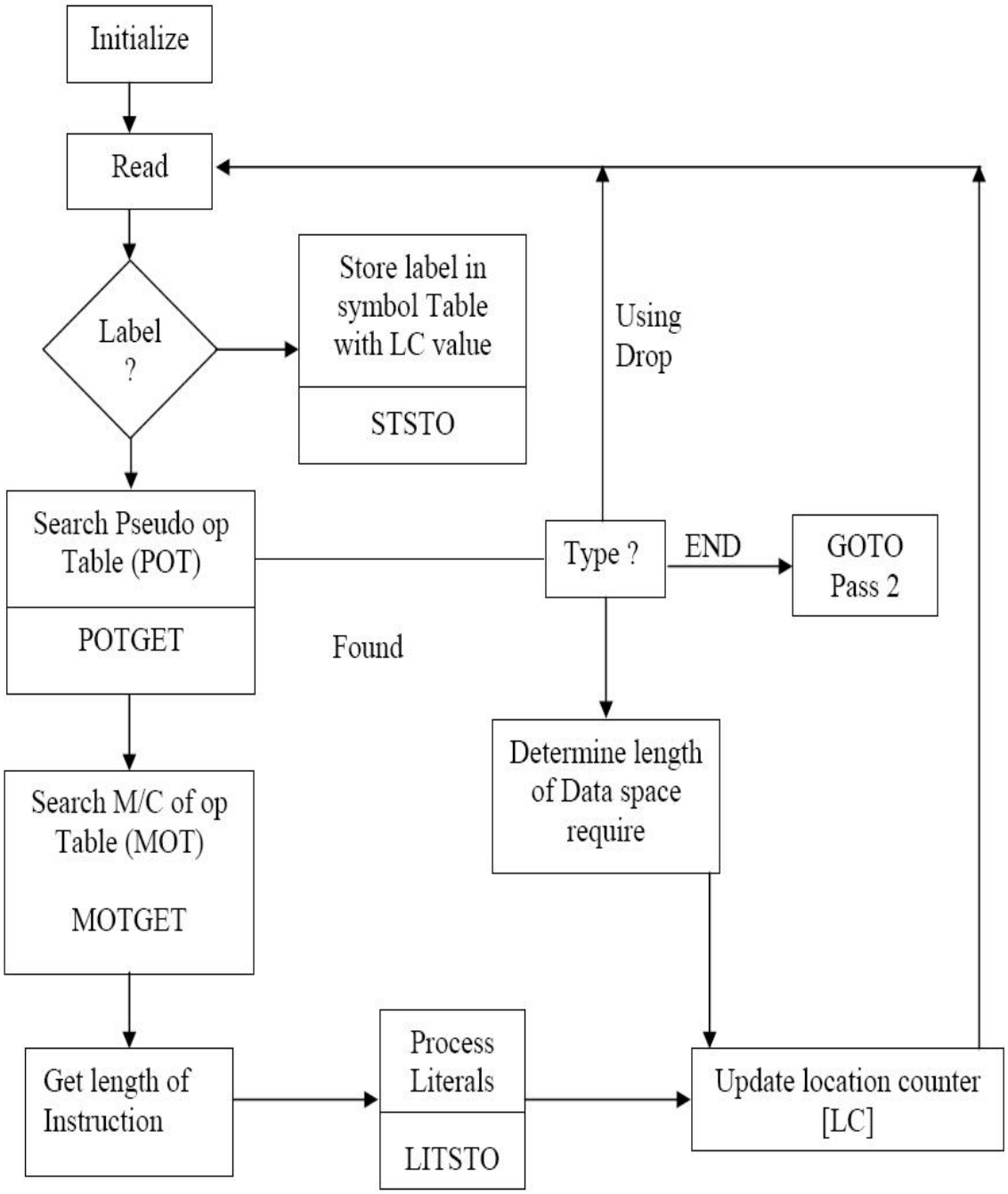
Codes:-

Availability:-

Y = register specified in USING pseudo-op

N = register never specified in USING pseudo-op or subsequently made unavailable by Drop pseudo-op.

Flow Chart of Pass 1 of Two Pass Assembler



Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 50

struct symbol{
    char name[20];
    int addr;
} sym[MAX];

struct literal{
    char name[20];
    int addr;
} lit[MAX];

int symCount=0, litCount=0;
int LC=0;

int searchSymbol(char s[]){
    for(int i=0;i<symCount;i++)
        if(strcmp(sym[i].name,s)==0)
            return 1;
    return 0;
}

void addSymbol(char s[]){
    strcpy(sym[symCount].name,s);
    sym[symCount].addr=LC;
```

```
    symCount++;
}

int searchLiteral(char s[]){
    for(int i=0;i<litCount;i++)
        if(strcmp(lit[i].name,s)==0)
            return 1;
    return 0;
}

void addLiteral(char s[]){
    char temp[20];

    if(strchr(s,'='))){
        char *p = strchr(s,'=');
        sprintf(temp,"='%c'", *(p+1));
        strcpy(lit[litCount].name,temp);
    }
    else{
        strcpy(lit[litCount].name,s);
    }

    lit[litCount].addr = -1;
    litCount++;
}

int main(){

    char label[20],opcode[20],operand[20];

    printf("Enter Assembly Code (use '-' if no label, END to finish)\n\n");
```

```
printf("LC\tLABEL\tOPCODE\tOPERAND\n");

while(1){

    scanf("%s %s %s",label,opcode,operand);

    if(strcmp(opcode,"START")==0){
        LC=atoi(operand);
        printf("%d\t%s\t%s\t%s\n",LC,label,opcode,operand);
    }

    else if(strcmp(opcode,"END")==0){

        printf("%d\t%s\t%s\t%s\n",LC,label,opcode,operand);

        for(int i=0;i<litCount;i++){
            lit[i].addr=LC;
            LC++;
        }
        break;
    }

    else if(strcmp(opcode,"DS")==0){

        if(strcmp(label,"-")!=0 && !searchSymbol(label))
            addSymbol(label);

        printf("%d\t%s\t%s\t%s\n",LC,label,opcode,operand);
        LC += atoi(operand);
    }

    else if(strcmp(opcode,"DC")==0){
```

```
        if(strcmp(label,"-")!=0 && !searchSymbol(label))
            addSymbol(label);

        printf("%d\t%s\t%s\t%s\n",LC,label,opcode,operand);
        LC++;
    }

    else{

        if(strcmp(label,"-")!=0 && !searchSymbol(label))
            addSymbol(label);

        if(strchr(operand,'=') && !searchLiteral(operand))
            addLiteral(operand);

        printf("%d\t%s\t%s\t%s\n",LC,label,opcode,operand);
        LC++;
    }
  }

  printf("\nSYMBOL TABLE\n");
  for(int i=0;i<symCount;i++)
      printf("%s\t%d\n",sym[i].name,sym[i].addr);

  printf("\nLITERAL TABLE\n");
  for(int i=0;i<litCount;i++)
      printf("%s\t%d\n",lit[i].name,lit[i].addr);

}
```

Output:

```
Enter Assembly Code (use '-' if no label, END to finish)

LC  LABEL   OPCODE  OPERAND
- START 100
- MOVER AREG,=5
LOOP ADD BREG,=1
- SUB AREG,=1
- BC ANY,LOOP
ONE DC 1
RESULT DS 1
- END -
100 -   START   100
100 -   MOVER   AREG,=5
101 LOOP    ADD BREG,=1
102 -   SUB AREG,=1
103 -   BC  ANY,LOOP
104 ONE DC  1
105 RESULT  DS  1
106 -   END -

SYMBOL TABLE
LOOP    101
ONE 104
RESULT  105

LITERAL TABLE
='5'    106
='1'    107
='1'    108
```

Conclusion:

1. Comment on working of Program by providing input source program.
2. Comment on databases generated during Pass 1 of Two Pass assembler.

Comment on working of Program by providing input source program.

The program implements a Two Pass Assembler. In Pass-1, the assembler reads the source program and assigns addresses to each instruction using the location counter. It identifies labels and stores them in the Symbol Table while also recording literals and generating intermediate code. For example, if the source program contains instructions like START 100,

MOVER AREG, ALPHA, ADD BREG, ONE, and STOP, the assembler processes each line and prepares necessary tables. In Pass-2, the assembler uses the intermediate code and tables generated in Pass-1 to replace symbolic operands with their actual addresses and produce the final machine code.

Comment on databases generated during Pass 1 of Two Pass assembler.

During Pass-1 of a Two Pass Assembler, several important databases are generated to assist in the translation process. The Symbol Table (SYMTAB) stores the symbols or labels along with their corresponding addresses. The Literal Table (LITTAB) maintains all literals used in the program and their assigned addresses. The Pool Table (POOLTAB) keeps track of the starting index of each literal pool. In addition, the assembler generates intermediate code and maintains the location counter (LC) to track the address of each instruction. These databases are later used in Pass-2 to generate the final machine code efficiently.